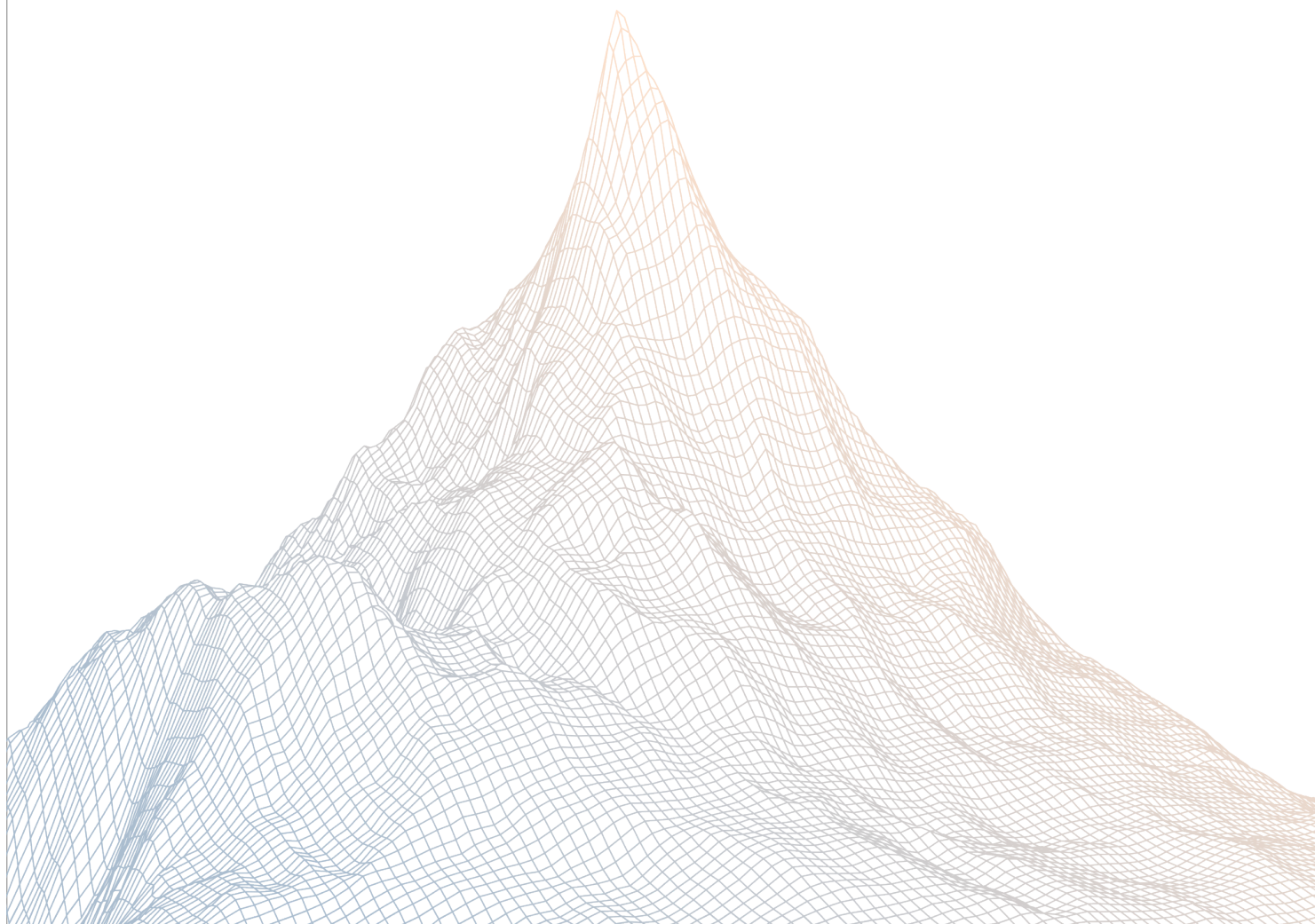


Berachain

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

June 25th to June 27th, 2025

AUDITED BY:

Mario Ponder
ether_sky

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Berachain	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Low Risk	7
4.2	Informational	11

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Berachain

Berachain is a high-performance EVM-Identical Layer 1 blockchain utilizing Proof-of-Liquidity (PoL) and built on top of the modular EVM-focused consensus client framework BeaconKit.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-internal
Repository	https://github.com/berachain/contracts-internal
Commit Hash	272b99328f0811ba4633cfddec802fec892989b
Files	Changes in the PR-10

2.3 Audit Timeline

June 25, 2025	Audit start
June 27, 2025	Audit end
July 5, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	4
Total Issues	7

3

Findings Summary

ID	Description	Status
L-1	setTargetRewardsPerSecond allows bypassing the rewards duration cooldown period	Resolved
L-2	Changing rewardsDuration mid-period leads to invalid getRewardForDuration return values	Resolved
L-3	The reward rate should be recalculated whenever targetRewardsPerSecond is updated	Acknowledged
I-1	Silent truncation of inputData.amountIn to uint160 in Permit2 approval	Resolved
I-2	Unbounded rewardsDuration in targetRewardsPerSecond-mode can result in excessively long reward periods	Acknowledged
I-3	The _setRewardsDuration function can be called even when the rewards duration remains unchanged	Resolved
I-4	Removing PERMIT2_EXPIRATION could simplify the logic	Resolved

4

Findings

4.1 Low Risk

A total of 3 low risk findings were identified.

[L-1] `setTargetRewardsPerSecond` allows bypassing the rewards duration cooldown period

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- src/pol/rewards/RewardVault.sol#L210-L247

Description:

The `setTargetRewardsPerSecond` function can be used to indirectly set the effective `rewardsDuration` without being subject to the cooldown period enforced by `setRewardsDuration`. By carefully selecting the `targetRewardsPerSecond` value before an incoming `notifyRewardAmount` transaction, the reward vault manager can achieve any desired `rewardsDuration` in the subsequent `_setRewardRate` call. This circumvents the intended restriction that reward duration changes should only occur after the cooldown period, as enforced in `setRewardsDuration`. As a result, the cooldown mechanism can be bypassed, potentially undermining protocol governance or user expectations.

Recommendations:

It is recommended to apply the same cooldown period check to `setTargetRewardsPerSecond` as is enforced in `setRewardsDuration`.

Berachain: Resolved with [@a48753ff920...](#)

Zenith: Verified. Fixed by removing the cooldown period from the `setRewardsDuration` function too.

[L-2] Changing rewardsDuration mid-period leads to invalid getRewardForDuration return values

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/base/StakingRewards.sol#L302-L304](#)
- [src/pol/rewards/RewardVault.sol#L210-L247](#)

Description:

The rewardsDuration variable can be changed during an ongoing reward period, either directly via `setRewardsDuration(_rewardsDuration)` or indirectly via `setTargetRewardsPerSecond(0)`. However, the `getRewardForDuration` view function simply returns a value based on the current rewardRate and the global rewardsDuration, without considering the actual duration of the current reward period. This can result in `getRewardForDuration` returning misleading or invalid values during the current period, which may impact integrators or users relying on this function for accurate information.

Recommendations:

It is recommended to introduce a `pendingRewardsDuration` variable to hold the next duration value and only apply the new duration within the `_setRewardRate` function, at the start of a new reward period.

Berachain: Resolved with [@a48753ff920...](#)

Zenith: Verified.

[L-3] The reward rate should be recalculated whenever `targetRewardsPerSecond` is updated

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [RewardVault.sol](#)

Description:

The reward vault manager can set `targetRewardsPerSecond`, which represents the desired reward rate per second.

- [RewardVault.sol#L246](#)

```
/// @inheritdoc IRewardVault
function setTargetRewardsPerSecond(uint256 _targetRewardsPerSecond)
    external onlyRewardVaultManager {
    emit TargetRewardsPerSecondUpdated(_targetRewardsPerSecond,
        targetRewardsPerSecond);
    targetRewardsPerSecond = _targetRewardsPerSecond;
}
```

However, this new rate isn't applied until the `notifyRewardAmount` function is called.

[https://github.com/berachain/contracts-](https://github.com/berachain/contracts-internal/blob/36900c93af93aadc6d6d3efec90c640f24e01400/src/pol/rewards/RewardVault.sol)

[internal/blob/36900c93af93aadc6d6d3efec90c640f24e01400/src/pol/rewards/RewardVault.sol](https://github.com/berachain/contracts-internal/blob/36900c93af93aadc6d6d3efec90c640f24e01400/src/pol/rewards/RewardVault.sol)

```
/// @inheritdoc IRewardVault
function notifyRewardAmount(bytes calldata pubkey, uint256 reward)
    external onlyDistributor {
    _notifyRewardAmount(reward);
    _processIncentives(pubkey, reward);
}
```

Of course, when updating the rewards duration using `setRewardsDuration` function, the change also doesn't take effect immediately. However, there's a difference between them. Setting a new `targetRewardsPerSecond` indicates a preferred reward rate that should ideally be applied promptly to align the protocol with the updated target.

Recommendations:

```
/// @inheritdoc IRewardVault
function setTargetRewardsPerSecond(uint256 _targetRewardsPerSecond)
    external onlyRewardVaultManager {
    emit TargetRewardsPerSecondUpdated(_targetRewardsPerSecond,
        targetRewardsPerSecond);
    targetRewardsPerSecond = _targetRewardsPerSecond;

    _notifyRewardAmount(0);
}
```

Berachain: Acknowledged, keeping same behaviour as duration mode where new changes applies only during next notifyRewards.

4.2 Informational

A total of 4 informational findings were identified.

[I-1] Silent truncation of `inputData.amountIn` to `uint160` in Permit2 approval

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/MetaAggregatorExecutor.sol#L56](#)

Description:

In `MetaAggregatorExecutor.execute`, the `inputData.amountIn` value is cast directly to `uint160` when calling `IPermit2(PERMIT2_ADDRESS).approve`. If `inputData.amountIn` exceeds `type(uint160).max`, this will silently truncate the value, potentially resulting in an insufficient approval amount. This could cause swaps to fail or behave unexpectedly, especially for tokens with very high decimals or large transfer amounts.

Recommendations:

It is recommended to use `toUint160` from `SafeCastLib` to safely cast and revert on overflow. Alternatively, add an explicit check and define a custom error `AmountInOverflow()`:
“solidity if (`inputData.amountIn` > `type(uint160).max`) revert `AmountInOverflow()`;”

Berachain: Resolved with [@ae303d3c438...](#)

Zenith: Verified.

[I-2] Unbounded rewardsDuration in targetRewardsPerSecond-mode can result in excessively long reward periods

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [src/pol/rewards/RewardVault.sol#L645-L664](https://github.com/Berachain/bera/blob/master/contracts/polynomial/RewardVault.sol#L645-L664)

Description:

In the RewardVault contract, when operating in targetRewardsPerSecond-mode, the rewardsDuration is dynamically calculated as $\text{undistributedRewards} / \text{targetRewardsPerSecond}$. If targetRewardsPerSecond is set to a very low value and there is a large amount of undistributedRewards (essentially $\text{undistributedRewards} \gg \text{targetRewardsPerSecond}$ whereby targetRewardsPerSecond can be as low as 1 wei per second), this calculation can result in an extremely large rewardsDuration that far exceeds the intended MAX_REWARD_DURATION. This, in turn, sets the periodFinish timestamp far into the future.

Recommendations:

Although the current design intentionally allows rewardsDuration to exceed MAX_REWARD_DURATION in this mode, it is recommended to introduce a reasonable upper limit for rewardsDuration when dynamically determining it in targetRewardsPerSecond-mode (e.g., ABSOLUTE_MAX_REWARD_DURATION).

Berachain: Acknowledged.

[I-3] The `_setRewardsDuration` function can be called even when the rewards duration remains unchanged

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [RewardVault.sol](#)

Description:

Whenever the `_setRewardsDuration` function is called, the `RewardsDurationUpdated` event is emitted. In the `_setRewardRate` function, if `_targetRewardsPerSecond` is high, the calculated `targetDuration` may be less than `minRewardDurationForTargetRate`. In such cases, `minRewardDurationForTargetRate` is used as the reward duration. Since `_setRewardRate` is called whenever a new reward is added, it's possible for the reward duration to remain unchanged across consecutive calls. However, `_setRewardsDuration` is still invoked each time, causing the `RewardsDurationUpdated` event to be emitted—even when the reward duration hasn't changed.

- [RewardVault.sol#L660](#)

```
function _setRewardRate() internal override {
    if (_targetRewardsPerSecond > 0) {
        uint256 targetDuration = undistributedRewards
        / _targetRewardsPerSecond;

        if (targetDuration < minRewardDurationForTargetRate) {
            _rewardRate = undistributedRewards
            / minRewardDurationForTargetRate;
            targetDuration = minRewardDurationForTargetRate;
        } else {
            // Use the target rate and update duration
            _rewardRate = _targetRewardsPerSecond;
        }
        _rewardsDuration = targetDuration;
        _setRewardsDuration(_rewardsDuration);
    }
}
```

Recommendations:

```
function _setRewardRate() internal override {
    if (_targetRewardsPerSecond > 0) {
        uint256 targetDuration = undistributedRewards
        / _targetRewardsPerSecond;

        if (targetDuration < minRewardDurationForTargetRate) {
            _rewardRate = undistributedRewards
        / minRewardDurationForTargetRate;
            targetDuration = minRewardDurationForTargetRate;
        } else {
            // Use the target rate and update duration
            _rewardRate = _targetRewardsPerSecond;
        }
        _rewardsDuration = targetDuration;

        if (_rewardsDuration != rewardsDuration) {
            _setRewardsDuration(_rewardsDuration);
        }
    }
}
```

Berachain: Resolved with [@a48753ff920...](#)

Zenith: Verified.

[I-4] Removing PERMIT2_EXPIRATION could simplify the logic

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [MetaAggregatorExecutor.sol](#)

Description:

When swapping uses the PERMIT2 contract, the expiration is currently set to 10 seconds.

- [MetaAggregatorExecutor.sol#L22C29-L22C47](#)

```
uint48 private constant PERMIT2_EXPIRATION = 10 seconds;

function execute(
    SwapInputData calldata inputData,
    SwapOutputData calldata outputData,
    AggregatorData calldata aggregatorData
)
    external
    returns (uint256 amountOut)
{
    if (inputData.isPermit2Approval) {
        inputData.tokenIn.safeApprove(PERMIT2_ADDRESS, inputData.amountIn);
        if (inputData.permit2SpenderAddress == address(0))
            revert ZeroAddress();
        IPermit2(PERMIT2_ADDRESS).approve(
            inputData.tokenIn,
            inputData.permit2SpenderAddress,
            uint160(inputData.amountIn),
            uint48(block.timestamp + PERMIT2_EXPIRATION)
        );
    } else {
        inputData.tokenIn.safeApprove(aggregatorData.aggregator,
            inputData.amountIn);
    }
    if (inputData.isPermit2Approval) {
        inputData.tokenIn.safeApprove(PERMIT2_ADDRESS, 0);
    }
}
```

```
    } else {  
        inputData.tokenIn.safeApprove(aggregatorData.aggregator, 0);  
    }  
}
```

However, since the swap and PERMIT2 approval occur within the same transaction and the approval is reset at the end, setting a 10-second expiration is unnecessary. Setting the expiration to the current time should be sufficient.

Recommendations:

```
uint48 private constant PERMIT2_EXPIRATION = 10 seconds;  
  
IPermit2(PERMIT2_ADDRESS).approve(  
    inputData.tokenIn,  
    inputData.permit2SpenderAddress,  
    uint160(inputData.amountIn),  
    uint48(block.timestamp + PERMIT2_EXPIRATION)  
    uint48(block.timestamp)  
);
```

Berachain: Resolved with [@ae303d3c438...](#)

Zenith: Verified.